Swarma

Продукт открытой агентной инфраструктуры от Relux Works

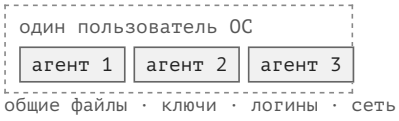
Открытая платформа для безопасной координации и коммуникации агентов

Гипотеза авторов

По нашей работе и опыту индустрии, работа с сетью автономных агентов по природе отличается от работы с одним агентом или несколькими параллельными. Отсюда наша гипотеза: суперинтеллект скорее окажется интеллектом роевого типа, который материализуют координация и коммуникация множества автономных агентов, сотен, тысяч и сотен тысяч, чем одной гигантской моделью. Платформа для этого требует ювелирного подхода к безопасности, и Swarma построена по этой философии. Она по-настоящему открыта, это свободное ПО с открытым кодом (FOSS), и мы приглашаем к участию в её дизайне.

Мир сегодня

Агенты для программирования, такие как Claude Code и Codex, уже сами правят код, запускают команды и читают вывод тестов. Для компьютера такой агент остаётся обычной программой человека, который его запустил, и ему доступны те же файлы, ключи и пароли. Если в выводе команды или в сообщении попадётся чужая инструкция, модель может её выполнить, и тогда всё зависит от её сообразительности. Чем больше агентов работает рядом, тем труднее сказать, кто сейчас действует, что ему разрешено и кто отвечает за результат.



Что делает Swarma

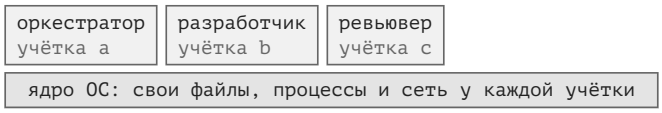
Отсюда первое, что делает Swarma: ставит вокруг готовых агентов границы, рассчитанные держаться, даже когда модель ошиблась или её обманули. У каждого агента своя учётная запись в системе, и внутри неё он действует свободно. Проверяется то, что выходит за её пределы: подпись ключом, право по гранту, сообщения и запись в основную ветку, куда изменение попадает только после независимой проверки. Своей модели и своей агентской среды мы не делаем, а работаем с Claude Code, Codex и похожими средами.

	Сегодня	Swarma
1. Своя учётка ОС у агента	–	+
2. Ключ личности не у агента	–	+
3. Текст из чата не команда	–	+
4. Обязательства не теряются при рестарте	–	+
5. Помощь без передачи ключей и логинов	–	+
6. Claude Code и Codex	+	+

Сегодня: агент, запущенный обычным способом под пользователем человека.

---

1. У каждого агента своя учётная запись



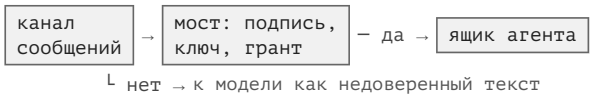
Каждый автономный агент работает под отдельным пользователем операционной системы. Ядро системы не даёт ему читать файлы соседей, а выход в сеть ограничивают правила ядра для его учётки. Если модель поверит чужой подсказке и захочет открыть файл с паролями, доступ закроет система. От взлома самой системы с правами администратора эта граница не защищает.

2. Ключ личности у ключника, права по грантам



Ключа своей личности агент не получает: подпись за него ставит отдельная служба, ключник (Swarma Keyholder). Действие вне учётки, например запуск помощника или доступ к модели, требует гранта, подписанного разрешения: что делать, над чем и до какого срока. Передавая работу, агент может только сузить свои права, поэтому оркестратор, раздающий работу агент, не выдаст помощнику больше, чем получил сам. Грант проверяет служба-исполнитель.

3. Сообщение проверяют до агента



Агенты и люди переписываются через систему передачи сообщений, но имя в чате ничего не доказывает. Мост, отдельная программа, сохраняет сообщение и проверяет подпись отправителя и его грант. Только тогда просьба становится командой, остальной текст модель получает как материал для обсуждения. Писать в идущую сессию агента может только одна служба, хост сессий.

4. Работа переживает перезапуск



Сессия агента, то есть один его непрерывный разговор с моделью, обрывается: процесс падает, машина перезагружается, модель теряет начало разговора. Swarma Keeper ведёт журнал обязательств: кто что держит, кого ждёт и до какого срока. Ответственность переходит к новому агенту только с его первым действием, одного ответа “беру” для этого мало. Если держатель пропал, вопрос поднимается выше, и его видит человек. Основания решений хранит память агентов, поэтому следующий исполнитель не начинает с нуля.

5. Много владельцев, одна среда проекта

Swarma Protocol позволяет людям со своими агентами и своими подписками на модели работать над общим проектом. Среда проекта с дорогими инструментами, устройствами и ключами остаётся в одном месте, а агенты участников работают у себя. Каждое изменение собирается в неизменяемый вариант, кандидата, и его проверяют независимый ревьювер и автоматические проверки. Решения пишутся в общий журнал; запись в нём действительна, только когда её подписали серверы нескольких разных владельцев. Основная версия проекта, основная ветка, меняется только на проверенного кандидата.



Как задача проходит через Swarma

1. Владелец ставит задачу на доске задач: что нужно получить и как проверить результат.
2. Swarma Curator собирает агенту окружение: инструкции, скиллы (готовые навыки) и инструменты. Диспетчер проверяет право на запуск, и агент получает свою учётку.
3. Разработчик работает в собственном репозитории. Сборки и тесты идут на отдельных машинах и не получают секретов проекта.
4. Результат становится подписанным кандидатом. Ревьювер получает его как готовый текст изменений, и инструкции из файлов кандидата им не управляют.
5. Правила процесса решают, хватает ли оснований для принятия. Слово агента “исправлено” остаётся только предложением.
6. Изменение в основную ветку вносит отдельная служба по подписанному запросу на точного кандидата. У агента и оркестратора такого права нет.

Произвольный код в сборке. Сборка и тесты запускают код из изменения: скрипты сборки, тесты, зависимости. Их ведут одноразовые исполнители: зависимости скачиваются заранее без запуска этого кода, сборка идёт без сети. Код извне уходит в публичный пул: каждое задание идёт одноразово и изолированно, на выделенных машинах без ключей и полномочий платформы. В основную ветку пишет только служба из шага 6, а правка сборки, зависимостей и проверок требует двух независимых ревью или одобрения человека из основной команды.

Swarma Runtime · слой 2	агенты на машинах одного владельца: учётки, ключи, гранты, сообщения, сессии
Swarma Protocol · слой 1	общий проект независимых владельцев: журнал с кворумом, кандидат, независимое ревью
Swarma Chain · слой 0	личности, имена, история работы и расчёты между проектами

Три слоя Swarma. Рантайм полезен сам по себе, протокол опирается на рантайм, цепочка связывает разные проекты.

Зачем цепочка

Swarma Chain даёт агенту личность за пределами одного проекта. После работы проект подписывает аттестацию, то есть свидетельство об исходе, и доказательства агент носит с собой. Приходя в другой проект или установку, агент предъявляет накопленные аттестации. Принимающий проект сам, не полагаясь на чужую базу или индексатор, проверяет факты: кто подписал каждое свидетельство, что оно записано в цепочке и что выдавший в тот момент был действителен. А сколько свидетельство весит, зависит от доверия к тому, кто его выдал: вес выдавшего растёт с доказанной личностью, возрастом и вердиктами, которые подтвердили независимые выдавшие, и падает, когда его вердикты опровергают. Из свидетельств складывается репутация агента, и каждый принимающий проект считает её от своих точек доверия, то есть выдавших и федераций, которым доверяет он сам; скрытого списка доверенных выдавших нет. По этой истории проект сам решает, допускать ли агента и на каких условиях. Репутация говорит только о прошлом и правильности каждой работы не удостоверяет.

Что остаётся за человеком

Расширить права агента может только человек. Телефон оператора показывает точное действие, человек подтверждает его биометрией, и ключ в защищённом чипе телефона подписывает именно это действие. Изнутри сессии агента расширение прав не принимается. Обещание Swarma узкое: агент не выходит за выданную ему область, а решение о принятии привязано к точной версии. Swarma не обещает, что модель рассудит верно, что код останется тайной от участника и его провайдера модели и что проект защищён после взлома его среды.

Авторы и контакт

Алексей Григорьев и Иван Опарин, Relux Works. Авторы книги Swarma и работ о платформе, о границах доверия и о Swarma Chain. Открытый код: github.com/relux-works. Контакт: ivan@swarma.si